quadsim: A Validated Quadrotor Sim-to-Real Reinforcement-Learning Platform on NVIDIA Isaac Lab

Technical Report

Drone_Sim_RL project

November 2026

Abstract

We present *quadsim*, a GPU-parallel quadrotor simulation and reinforcement-learning (RL) platform built on Isaac Lab v2.3.2 and Isaac Sim 5.1. The platform couples a framework-agnostic PyTorch model of multirotor physics (asymmetric first-order motors, rotor and body drag, gyroscopic and reaction torques, ground effect, wind) with PhysX rigid-body integration, models the onboard sensing chain (IMU, barometer, state estimate, depth/RGB cameras, range sensing) including noise, bias and latency, and exposes hover, trajectory tracking, waypoint and vision-based obstacle-avoidance tasks with deployable observations only. Every component is validated against analytic solutions or an independent integrator: PhysX trajectories match the reference integrator to 0.4 mm over 1.5 s of open-loop flight, RTX-rendered depth matches our analytic GPU ray caster to 7 mm median error, and the exported ONNX policy, driven by a NumPy deployment interface, reproduces simulator observations and actions to 10^{-6} in closed loop. PPO policies trained in minutes on an 8 GB consumer GPU outperform classical cascade and geometric controllers under identical noise, latency and randomization. We report what is verified and, explicitly, what is not (no hardware or PX4-SITL flights yet), and give a prioritized roadmap toward real-world deployment.

Contents

1 Introduction and scope **3**

2 Framework selection and related work **3**

3 System architecture **3**

4 Mathematical models **4**
4.1 Frames and rigid-body dynamics 4
4.2 Rotors and motors 4
4.3 Aerodynamics 4
4.4 Wind and disturbances 5
4.5 Control allocation and classical controllers 5
4.6 Sensor models 5

5 Reinforcement-learning environments **5**
5.1 Observation and action design 5
5.2 Tasks and rewards 6
5.3 Domain randomization 6
5.4 Learning algorithm 6

6 Validation and experiments **6**
6.1 M1: simulator foundation 7
6.2 M2: quadrotor dynamics 7
6.3 M3: sensors, recording and visualization 7
6.3.1 Upstream issue found and patched 8
6.4 M4: state-based reinforcement learning 9
6.5 M6: export and deployment pipeline 9

7 Limitations **10**

8 Roadmap toward real-world deployment **10**

9 Reproducibility **11**

1 Introduction and scope

Learning-based quadrotor control has reached champion-level performance in drone racing [1] and robust low-level control learned in seconds [2]. Such results rest on simulators whose physics and sensing are close enough to reality, and on disciplined interfaces between policy and vehicle. The goal of this project is a *research-grade* platform with these properties, prioritizing verified functionality over feature count.

Hardware note. The original brief targeted an RTX 5070 (12 GB). The actual machine has an **RTX 5060 with 8 GB** (Blackwell, sm_120), Ubuntu 24.04, an i5-14600KF and 32 GB RAM. All budgets (env counts, image sizes) were chosen for 8 GB.

Validation policy. Each milestone has measurable acceptance criteria that are executed by scripts in `scripts/validation`; results are stored as JSON in `docs/results` and summarized in `docs/PROGRESS.md`. RL results distinguish *training execution* from *demonstrated policy performance*, which is always measured by a separate deterministic evaluation with confidence intervals.

2 Framework selection and related work

Isaac Lab offers GPU-parallel PhysX simulation, tiled RTX rendering and RL integrations. At the time of writing the latest releases were v2.3.2 (Isaac Sim 5.1) and v3.0.0 (Isaac Sim 6.2). We selected **v2.3.2 + Isaac Sim 5.1**: it is the latest stable pair, it is available as an official container (`nvcr.io/nvidia/isaac-lab:2.3.2`), and v3.0 introduces breaking changes (quaternion convention $wxyz \to xyzw$, multi-backend refactor). The physics core of quadsim is framework-agnostic to ease a later migration.

Existing quadrotor components. Isaac Lab's `Isaac-Quadcopter-Direct` task applies a body wrench directly from actions without motor dynamics; the `isaaclab_contrib` multirotor asset (from the Aerial Gym authors [4]) adds a first-order thruster model with an allocation matrix. Aerial Gym [4] and OmniDrones [5] provide broader multirotor suites on Isaac Gym / Isaac Sim. We reused ideas (asymmetric motor time constants, allocation matrices, depth-based navigation in cluttered environments) but implemented our own model to obtain (i) rotor drag, gyroscopic, reaction and ground-effect terms, (ii) unit-testability without a simulator, and (iii) an independent reference integrator for validation. The vehicle parameters follow the PX4 SITL Gazebo model of the Holybro X500 so that results can be cross-checked against PX4 SITL later.

Network constraint. NVIDIA's asset CDN (Nucleus/S3) was effectively unreachable from the development network (the stock quadcopter task did not finish loading its USD within 625 s), and NGC refused direct container pulls (HTTP 403). Consequently all assets are *procedural* and the container base image is fetched through the user's proxy by a small registry client with digest verification.

3 System architecture

The code base is layered (Fig. 1):

- `quadsim.core` (pure PyTorch): parameters, motor/aerodynamic model, reference integrators, controllers, sensor and disturbance models, analytic ray casting (PyTorch and NVIDIA Warp kernels).
- `quadsim.isaac`: procedural USD quadrotor (explicit `MassAPI`: mass, inertia and CoM are written from the parameters, not derived from geometry) and version-gated runtime patches for upstream issues.
- `quadsim.tasks`: Isaac Lab `DirectRLEnv` base environment and tasks, PPO configurations and classical baselines that act through the same action interface.

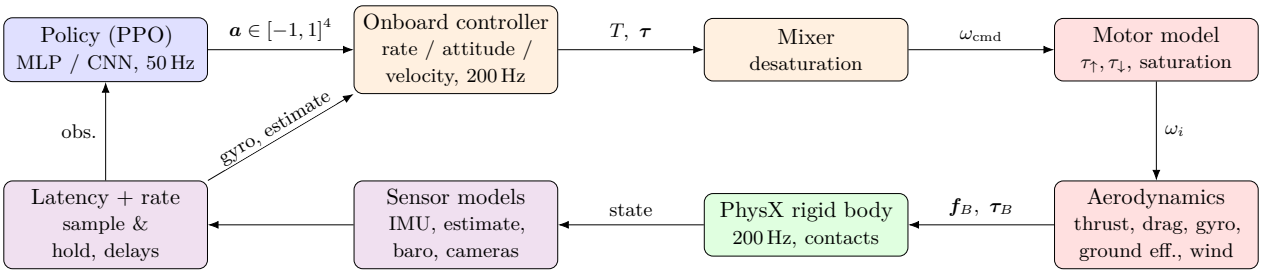


Figure 1: Per-step data flow. Orange, red and violet blocks belong to the framework-agnostic PyTorch core (`quadsim.core`) and are unit-tested without Isaac Sim; the green block is PhysX (Isaac Sim). Rendering (RTX cameras) is an alternative source of image observations.

- `quadsim.rl`, `quadsim.data`, `quadsim.deploy`: networks and policy I/O, HDF5 recording, export and the NumPy deployment interface.

Isaac Lab itself is not modified; it is installed from a pinned tag into `third_party/`.

Simulation timeline. Physics runs at 200 Hz, the policy at 50 Hz (decimation 4). Within each physics step: sensors are sampled; the delayed policy command is passed to the onboard controller (which uses the *noisy* gyro and state estimate); the mixer produces rotor-speed commands; motor states are advanced exactly; the aerodynamic wrench is evaluated and written to PhysX through Isaac Lab's wrench composer (body frame); PhysX integrates the rigid body and contacts.

4 Mathematical models

4.1 Frames and rigid-body dynamics

The world frame W is z-up (ENU-like), the body frame B is FLU, and $\boldsymbol{q} \in S^3$ (scalar first) rotates B into W with rotation matrix $\boldsymbol{R}(\boldsymbol{q})$. With mass m and principal inertia $\boldsymbol{J} = \mathrm{diag}(J_x, J_y, J_z)$,

$$\dot{\boldsymbol{p}} = \boldsymbol{v}, \qquad \dot{\boldsymbol{v}} = \tfrac{1}{m}\boldsymbol{R}\,\boldsymbol{f}_B + \boldsymbol{g} + \tfrac{1}{m}\boldsymbol{f}_W^{\text{ext}}, \tag{1}$$

$$\dot{\boldsymbol{q}} = \tfrac{1}{2}\,\boldsymbol{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix}, \qquad \boldsymbol{J}\dot{\boldsymbol{\omega}} = \boldsymbol{\tau}_B - \boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega}, \tag{2}$$

with $\boldsymbol{g} = (0, 0, -9.81)$. PhysX integrates these equations with a symplectic (semi-implicit) Euler scheme and gyroscopic forces enabled; the reference integrator offers RK4 and the same semi-implicit scheme.

4.2 Rotors and motors

Rotor i at lever arm $\boldsymbol{r}_i$ (relative to the CoM) with spin $s_i = +1$ (CCW) or -1 spins at ω_i:

$$\dot{\omega}_i = \frac{\omega_{\text{cmd},i} - \omega_i}{\tau_i}, \qquad \tau_i = \begin{cases} \tau_\uparrow & \omega_{\text{cmd},i} \geq \omega_i \\ \tau_\downarrow & \text{otherwise} \end{cases}, \qquad \omega_{\text{cmd},i} \in [\omega_{\min}, \omega_{\max}], \tag{3}$$

discretized exactly under zero-order hold: $\omega_i^+ = \omega_{\text{cmd}} + (\omega_i - \omega_{\text{cmd}})e^{-\Delta t/\tau_i}$. Thrust and reaction torque are $T_i = k_f\,\omega_i^2\,g_e(h_i)$ and $Q_i = -s_i\,(k_m T_i + J_r\dot{\omega}_i)$, the latter including the spin-up reaction of the rotor inertia J_r.

4.3 Aerodynamics

With air-relative velocity $\boldsymbol{v}_{\text{rel}} = \boldsymbol{v}_B - \boldsymbol{w}_B$ (wind $\boldsymbol{w}$) and its component $\boldsymbol{v}_{\perp,i}$ orthogonal to the rotor axis at the hub ($\boldsymbol{v}_{\text{rel}} + \boldsymbol{\omega} \times \boldsymbol{r}_i$),

$$\boldsymbol{f}_{\text{rd},i} = -c_{\text{rd}}\,\omega_i\,\boldsymbol{v}_{\perp,i} \quad \text{(blade flapping / induced drag)}, \qquad \boldsymbol{m}_{\text{rm},i} = -c_{\text{rm}}\,s_i\,\omega_i\,\boldsymbol{v}_{\perp,i}, \tag{4}$$

$$\boldsymbol{f}_{\text{body}} = -\boldsymbol{D}_1\boldsymbol{v}_{\text{rel}} - \boldsymbol{D}_2\,|\boldsymbol{v}_{\text{rel}}| \odot \boldsymbol{v}_{\text{rel}}, \qquad \boldsymbol{\tau}_{\text{gyr}} = -\boldsymbol{\omega} \times \textstyle\sum_i J_r s_i \omega_i\,\boldsymbol{e}_z, \tag{5}$$

and the in-ground-effect gain of Cheeseman and Bennett, $g_e(h) = \left(1 - (R/4h)^2\right)^{-1}$, capped at $h \geq R/2$. The total body wrench is

$$\boldsymbol{f}_B = \sum_i \left(T_i \boldsymbol{e}_z + \boldsymbol{f}_{\mathrm{rd},i}\right) + \boldsymbol{f}_{\mathrm{body}}, \quad \boldsymbol{\tau}_B = \sum_i \left[\boldsymbol{r}_i \times \left(T_i \boldsymbol{e}_z + \boldsymbol{f}_{\mathrm{rd},i}\right) + \boldsymbol{m}_{\mathrm{rm},i} + Q_i \boldsymbol{e}_z\right] + \boldsymbol{\tau}_{\mathrm{gyr}}. \tag{6}$$

The X500 preset uses the PX4 Gazebo values ($k_f = 8.549{\times}10^{-6}$, $k_m = 0.016$, $\tau_\uparrow = 12.5\,\mathrm{ms}$, $\tau_\downarrow = 25\,\mathrm{ms}$, $\omega_{\max} = 1000\,\mathrm{rad/s}$, $c_{\mathrm{rd}} = 8.06 \times 10^{-5}$); its composite mass and inertia (base + rotors, parallel-axis theorem) are $m = 2.064\,\mathrm{kg}$, $\boldsymbol{J} = \mathrm{diag}(0.0239, 0.0239, 0.0440)\,\mathrm{kg\,m^2}$, hover speed $769.6\,\mathrm{rad/s}$ and thrust-to-weight 1.69. Body drag coefficients are engineering estimates (not identified).

4.4 Wind and disturbances

Wind is $\boldsymbol{w} = \bar{\boldsymbol{w}} + \boldsymbol{w}_{\mathrm{t}} + \boldsymbol{w}_{\mathrm{g}}$: a per-episode mean, Gauss–Markov turbulence $\boldsymbol{w}_{\mathrm{t}}^{+} = a\boldsymbol{w}_{\mathrm{t}} + \sigma\sqrt{1-a^2}\,\boldsymbol{\eta}$ with $a = e^{-\Delta t/\tau_w}$ (exact discretization of a Dryden-like first-order process with stationary standard deviation σ) and discrete $1-\cos$ gusts (MIL-F-8785C shape) with Poisson arrivals. Push disturbances apply random finite-duration forces.

4.5 Control allocation and classical controllers

The allocation $[F_z, \tau_x, \tau_y, \tau_z]^\top = \boldsymbol{A}\boldsymbol{T}$ with rows $(1,\, y_i,\, -x_i,\, -s_i k_m)$ is inverted with PX4-style sequential desaturation: roll/pitch are scaled only if their spread exceeds the actuator range, the collective is shifted to preserve roll/pitch authority, and yaw is added with the largest feasible scale. The onboard rate controller is $\boldsymbol{\tau} = \boldsymbol{J}(\boldsymbol{K}_p \boldsymbol{e}_\omega + \boldsymbol{K}_i \int \boldsymbol{e}_\omega - \boldsymbol{K}_d \dot{\boldsymbol{\omega}}_f) + \boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega}$ (derivative on the filtered measurement), the attitude controller is a quaternion P law on the rotation vector of $\boldsymbol{q}^{-1} \otimes \boldsymbol{q}_{\mathrm{sp}}$, and the position controller is a P–PI cascade producing a thrust vector with acceleration and tilt limits. The geometric baseline uses Lee et al.'s SE(3) law [6] $\boldsymbol{f} = m(-\boldsymbol{K}_x \boldsymbol{e}_x - \boldsymbol{K}_v \boldsymbol{e}_v + g\boldsymbol{e}_3 + \ddot{\boldsymbol{p}}_d)$.

4.6 Sensor models

IMU: $\tilde{\boldsymbol{\omega}} = (\boldsymbol{I} + \boldsymbol{M}_g)\boldsymbol{\omega} + \boldsymbol{b}_g + \boldsymbol{n}_g$, $\tilde{\boldsymbol{a}} = (\boldsymbol{I} + \boldsymbol{M}_a)\boldsymbol{R}^\top(\dot{\boldsymbol{v}} - \boldsymbol{g}) + \boldsymbol{b}_a + \boldsymbol{n}_a$, where the specific force is obtained by finite differences of the PhysX velocity (exact for the symplectic scheme), white noise has density σ_c (discrete std $\sigma_c/\sqrt{\Delta t_s}$), and biases are a per-episode turn-on value plus a Gauss–Markov drift with random-walk intensity σ_{rw} (stationary std $\sigma_{rw}\sqrt{\tau/2}$). Default values are the PX4/rotors SITL IMU parameters. Measurements are sampled and held at the sensor rate and pass a per-episode random delay. **State estimate:** position (white + random-walk drift), velocity and attitude (multiplicative rotation-vector) noise with latency, emulating EKF/VIO/mocap outputs. **Barometer:** ISA conversion with noise and Gauss–Markov bias. **Depth camera:** distance-to-image-plane with stereo-like noise $\sigma(z) = \sigma_0 + kz^2$, quantization, per-pixel dropouts, minimum/maximum range; no-return pixels read as maximum range. **Analytic ray casting:** exact ray–plane, ray–finite-cylinder (side and cap) and ray–oriented-box intersections, implemented in PyTorch and as an NVIDIA Warp kernel with $O(NR)$ memory.

5 Reinforcement-learning environments

5.1 Observation and action design

The `policy` observation contains only quantities available onboard: estimated body-frame velocity, the first two columns of the estimated rotation matrix (continuous attitude representation), gyro rates, the previous action and task information expressed in the estimated body frame. The `critic` additionally receives the true state, normalized rotor speeds, wind, disturbance force, and normalized domain-randomization parameters (asymmetric actor-critic [7]). Four action interfaces are available: individual rotor thrusts, collective thrust and body rates (CTBR, the default for agile tasks following [3]), collective thrust and attitude, and velocity commands (default for navigation, matching PX4 offboard velocity control).

5.2 Tasks and rewards

All rewards are integrated over the policy step Δt; crashes (ground contact, excessive tilt, velocity, distance, or obstacle collision) terminate the episode with a penalty.

- **Hover/recovery**: initial states up to $\pm 1.5\,\mathrm{m}$ offset, 57° tilt, $1.5\,\mathrm{m/s}$, $3\,\mathrm{rad/s}$; reward $1 - \tanh(\|e_p\|/0.8) + 0.5\,e^{-(\|e_p\|/0.1)^2} + 0.2\cos^2$-type yaw term minus velocity, body-rate and action-rate penalties.
- **Tracking**: references $\boldsymbol{p}_d(t) = \boldsymbol{c} + \sum_k \boldsymbol{A}_k \sin(\boldsymbol{\omega}_k t + \boldsymbol{\phi}_k)$ covering circles, figure-eights and random smooth curves (peak speed $\leq 4\,\mathrm{m/s}$); the policy observes the reference at $t + \{0, 0.1, 0.2, 0.4, 0.6, 1.0\}\,\mathrm{s}$.
- **Waypoints**: five random waypoints (separation $\geq 1.5\,\mathrm{m}$); progress reward $\|\boldsymbol{p} - \boldsymbol{w}\|_{t-1} - \|\boldsymbol{p} - \boldsymbol{w}\|_t$, reach bonus, time penalty.
- **Obstacle navigation**: up to 24 kinematic cylinders ($r \in [0.15, 0.45]\,\mathrm{m}$) rejection-sampled with a minimum gap of $0.9\,\mathrm{m}$, goal 11–12 m ahead; progress, goal, heading-alignment (camera looks where the vehicle flies), altitude-band and proximity terms. A curriculum increases the maximum obstacle count from 8 to 24 when the running success rate exceeds 70 %. Observation variants: 32-ray range ring, analytic depth (64×48, HFOV 87°), RTX depth, RGB, RGB-D.

5.3 Domain randomization

Startup (written to PhysX): mass $\times[0.9, 1.1]$, inertia $\times[0.85, 1.15]$ per axis, CoM offset $\pm 1\,\mathrm{cm}$. Per episode (model side): $k_f \times [0.9, 1.1]$ per rotor, $k_m \times [0.85, 1.15]$, motor time constants $\times[0.7, 1.6]$, $\omega_{\max} \times [0.92, 1]$ (battery sag), drag $\times[0.5, 1.5]$, payload $-5 \ldots + 10\,\%$ of weight, sensor-noise scale $\times[0.5, 1.5]$, action delay 0–2 and estimator delay 0–2 physics steps.

5.4 Learning algorithm

PPO [8] (rsl_rl 3.1.2) with GAE ($\gamma = 0.99, \lambda = 0.95$), adaptive learning rate (KL target 0.01), 32 steps per environment, empirical observation normalization. State policies: actor MLP 256–256–128, critic 512–256–128 (ELU). Vision policies: a 3-layer CNN encoder (32/64/64 channels) to a 128-d feature, fused with the normalized state in a 256–128 actor; the critic is a state-only MLP on privileged inputs.

6 Validation and experiments

Table 1 summarizes the acceptance status of every milestone; the subsections give the evidence.

Milestone	Acceptance criterion (executed)	Status
M1 Isaac Sim/Lab	headless GPU boot; PhysX free fall vs symplectic Euler $< 1\,\mathrm{mm}$ (got $9.5 \times 10^{-7}\,\mathrm{m}$)	**pass**
M2 Dynamics	46+ unit tests; PhysX vs reference $< 1\,\mathrm{cm}/1.5\,\mathrm{s}$; inertia exact; motor/yaw models $< 3\,\%$	**pass**
M3 Sensors, recording, visualization	sensor statistics; rendered vs analytic depth (median $< 2\,\mathrm{cm}$); HDF5 round trip; GUI and video	**pass**
M4 State-based RL	training completes with checkpoints; evaluated vs classical baselines	**partial**
M5 Vision RL	depth policy trained and evaluated; RGB/RGB-D pipelines execute	**partial**
M6 DR & Sim-to-Real interfaces	DR ablation; ONNX = PyTorch to 10^{-5}; deployment-in-the-loop; ROS 2/PX4 bridge	**partial**
M7 Docker, benchmark, docs	container runs tests + GPU smoke test; benchmark table; README; report	**pass**

Table 1: Milestone acceptance summary.

6.1 M1: simulator foundation

Sixty-four rigid bodies in free fall for 0.5 s at $\Delta t = 2.5$ ms end at $|z - z_{\text{symplectic}}| = 9.5 \times 10^{-7}$ m (float32 resolution) and $|z - z_{\text{analytic}}| = 6.1$ mm, which is exactly the expected first-order bias $g\,t\,\Delta t/2$; this identifies PhysX's integrator as symplectic Euler, which the reference simulator reproduces. The same test passes inside the Docker container.

6.2 M2: quadrotor dynamics

Unit tests (CPU, no simulator) verify quaternion algebra, FLU sign conventions of roll/pitch/yaw torques, the exact motor step response (63.2 % after τ, separate rise/fall constants, discretization invariance), rotor and body drag, ground effect and gyroscopic torque, analytic free fall, constant-torque yaw, conservation of angular momentum and energy for a tumbling body (Dzhanibekov effect), first-order convergence of the semi-implicit integrator to RK4, closed-loop position steps with the full cascade (< 5 cm), recovery of the Crazyflie from 46° tilt and 3.7 rad/s, SE(3) circle tracking (< 10 cm RMSE), and allocation/desaturation properties. Table 2 lists the Isaac Sim integration tests; Fig. 2 shows the open-loop comparison.

Test	X500	Crazyflie	Criterion
PhysX mass / inertia rel. error	0e+00 / 0e+00	0e+00 / 0e+00	$< 10^{-4}$
Hover altitude drift over 2 s [mm]	0.00	0.00	< 5
Thrust step: accel. RMS err / peak	0.019 %	0.026 %	< 2 %
Yaw accel. vs τ_z/J_z	0.0036 %	0.0077 %	< 3 %
Open loop 1.5 s, pos. err vs semi-implicit ref. [mm]	0.40	2.07	< 10
Open loop, att. err vs semi-implicit ref. [deg]	0.006	0.066	< 0.5
Open loop, pos. err vs RK4 ref. [mm] (PhysX discretization)	3.3	8.9	info
Closed-loop velocity cascade, PhysX vs ref. [mm]	12.3	9.2	< 30

Table 2: PhysX integration of the quadrotor model vs analytic models and the independent reference integrator (`scripts/validation/m2_dynamics_validation.py`). Sensor noise, latency and randomization disabled.

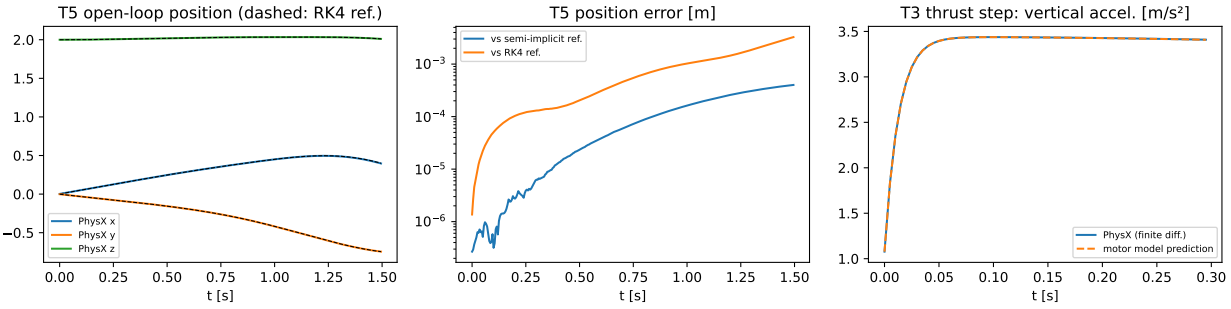


Figure 2: X500: open-loop multisine rotor excitation (left), position error of PhysX vs the reference integrator (middle; semi-implicit reference = same discretization, RK4 = integration error of PhysX at 200 Hz) and thrust-step response (right).

6.3 M3: sensors, recording and visualization

Statistical unit tests confirm the IMU white-noise standard deviation at 200 Hz ($\sigma_c\sqrt{f}$, within 5 %), turn-on bias, exact latency, zero specific force in free fall and the correct gravity direction when tilted, ISA barometric conversion (100 m → 100 129 Pa), Gauss–Markov stationary statistics, depth-noise statistics, exact ray–primitive intersections and agreement of the Warp ray-casting kernel with the

PyTorch implementation. A recording plot revealed a bug (no-return depth pixels received $\sigma \propto z^2$ noise); it was fixed and a regression test added.

Rendered vs analytic depth. For 64 random poses among 24 obstacles (96×72, HFOV 87°, noise off), RTX-rendered and analytic depth agree as in Table 3. Residuals concentrate on silhouette pixels (anti-aliasing) and cylinder tessellation. The comparison also exposed that the initial camera mount saw the X500's front rotor discs (3 % of the image); the default mount was moved to $(0.15, 0, -0.06)$ m, which has no self-occlusion at zero pitch.

Camera pitch	median [mm]	p90 [cm]	hit/miss agree	self-occl.	pixels
0°	7.2	2.4	99.6 %	0.0 %	337361
15°	9.0	2.3	99.5 %	5.6 %	254392

Table 3: RTX TiledCamera depth vs analytic ray-cast depth (`scripts/validation/m3_depth_consistency.py`). Self-occluded pixels (vehicle rotors, rendered < 0.4 m, analytic > 0.45 m) are excluded from the error metrics and reported separately.

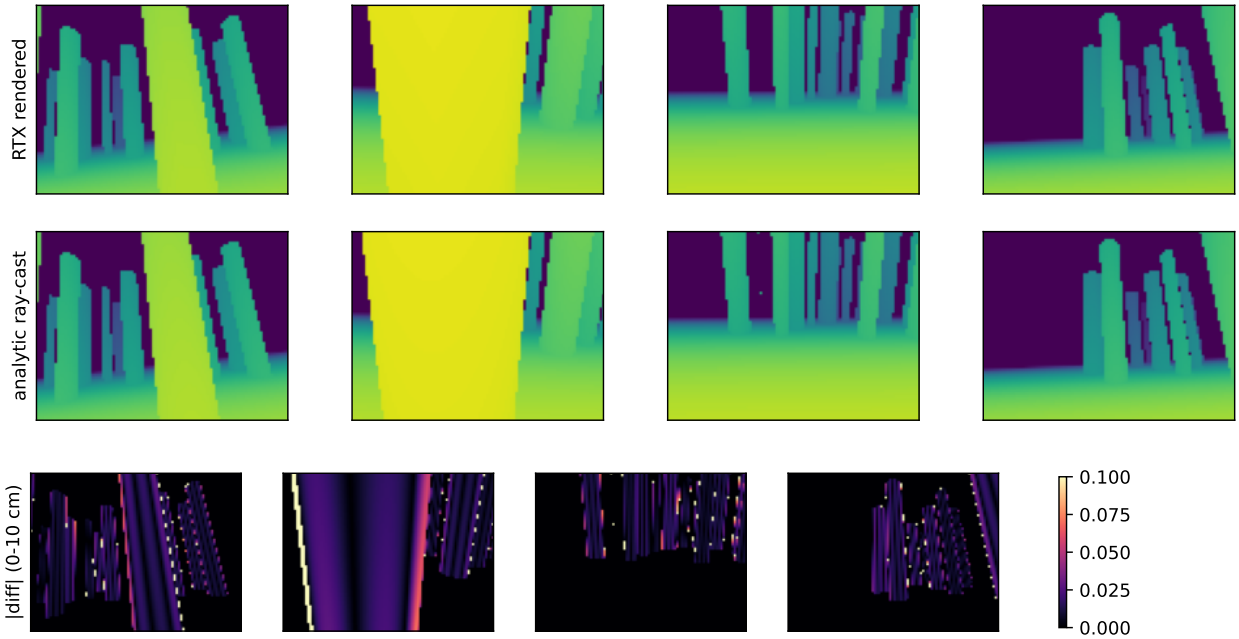


Figure 3: Rendered depth (top), analytic ray-cast depth (middle) and absolute difference clipped at 10 cm (bottom).

Recording. The synchronized HDF5 recorder was verified by round trips for a state task (tracking, SE(3) baseline) and a rendered-depth task (navigation, potential-field baseline): dataset shapes, monotonic timestamps with constant Δt, exact equality of the last sample with the live simulator, and the embedded configuration. **Visualization.** Headless MP4 recording and the interactive GUI viewer (chase camera, target and reference markers) work after patching an upstream Isaac Lab issue (Section 6.3.1).

6.3.1 Upstream issue found and patched

With any rendered viewport (GUI, `-video`), every RigidObject failed to initialize with "Failed to get rigid body transforms from backend" — reproduced with a plain Isaac Lab cuboid. Cause: `WrenchComposer.__init__` uses `hasattr(asset.data, "body_link_pos_w")`; `hasattr` evaluates the property, which queries PhysX before its GPU buffers exist during the PLAY callback, and the raised `Exception` is not an `AttributeError`. quadsim applies a version-gated patch that checks the attribute on the class (no PhysX query), leaving Isaac Lab's sources untouched. Isaac Lab's viewport

chase camera has a similar start-up read, which the viewer avoids by enabling asset following only after the first reset.

6.4 M4: state-based reinforcement learning

All policies use CTBR actions at 50 Hz, the full randomization of Section 5.3, sensor noise and latency. Evaluation uses the deterministic policy and identical seeds (hence identical initial states and randomization) for all controllers; ± denotes a 95 % confidence interval of the mean.

Hover and recovery. Training: 4096 environments, 600 PPO iterations (79 M transitions, ≈10 min on the RTX 5060). Success requires $\|e_p\| < 10$ cm and $\|v\| < 0.2$ m/s at the end of the 8 s episode. The policy also generalizes to wind (0–3 m/s mean, turbulence, gusts) and push disturbances that were *not* present during training (Table 4). The policy's yaw accuracy (4.4°) is worse than the baseline's because of the weak yaw reward weight.

Condition	Controller	success [%]	crash [%]	final pos. err [cm]	final yaw err [deg]	episodes
nominal	PPO (ours)	99.9 ± 0.2	0.1 ± 0.2	1.38 ± 0.26	4.4 ± 0.2	1000
nominal	Cascade PID	88.7 ± 2.0	0.5 ± 0.4	6.10 ± 0.74	1.5 ± 0.3	1000
wind + pushes	PPO (ours)	97.3 ± 1.0	0.1 ± 0.2	2.86 ± 0.33	4.6 ± 0.2	1000
wind + pushes	Cascade PID	78.6 ± 2.5	0.7 ± 0.5	8.27 ± 0.94	1.8 ± 0.4	1001

Table 4: Hover/recovery from up to 57° tilt, 3 rad/s, 1.5 m/s and 1.5 m offset (1000 episodes per row).

Trajectory tracking. Training: 4096 environments, 1500 iterations (197 M transitions). The aggressive reference set contains accelerations up to ≈15 m/s^2, partly beyond the X500's sustainable 13 m/s^2 (thrust-to-weight 1.69) and beyond the baselines' tilt limits; the moderate set (≤2 m/s, ≤0.35 Hz) is feasible for all controllers. The first SE(3) baseline, which mapped Lee's torque-level attitude law onto rate set-points at the 50 Hz policy rate, was unstable (53 % crashes, row †); the redesigned baseline uses Lee's position law with the onboard attitude/rate loops.

Reference set	Controller	RMSE [cm]	max err [cm]	crash [%]	episodes
aggressive (≤4 m/s)	PPO (ours)	12.6 ± 1.5	40.6 ± 3.0	0.2 ± 0.4	500
aggressive (≤4 m/s)	Cascade PID	46.6 ± 3.7	102.8 ± 7.2	3.2 ± 1.5	500
aggressive (≤4 m/s)	SE(3) geometric	49.2 ± 3.9	97.7 ± 6.9	1.8 ± 1.2	500
aggressive (≤4 m/s)	SE(3) v1 (torque-level @50 Hz)†	114.2 ± 5.0	221.8 ± 9.0	53.2 ± 4.4	500
moderate (≤2 m/s)	PPO (ours)	5.8 ± 0.2	29.0 ± 0.8	0.0 ± 0.0	500
moderate (≤2 m/s)	Cascade PID	17.4 ± 1.3	50.2 ± 3.7	0.0 ± 0.0	500
moderate (≤2 m/s)	SE(3) geometric	22.7 ± 1.4	52.8 ± 3.8	0.2 ± 0.4	500

Table 5: Trajectory tracking (500 episodes of 10 s per row). Maximum errors include the initial transient (start offset up to 0.3 m).

6.5 M6: export and deployment pipeline

The exported ONNX graph (normalizer + actor + clamp) matches PyTorch to 3.6×10^{-7}. In the deployment-in-the-loop test the simulator is flown by the ONNX policy whose observation is rebuilt every step by the NumPy `ObservationBuilder` from onboard quantities only. Over 512 hover episodes the maximum observation error is 9.5×10^{-7}, the maximum action error 1.0×10^{-6}, and the success rate (99.8 %) and final error (1.29 cm) equal those of the PyTorch policy. Frame conversions to PX4 (NED/FRD) are unit-tested, including the consistency of a physical body vector through both quaternion conventions.

7 Limitations

The following limitations are explicit; none of them is hidden by the results above.

- **No real-world or PX4-SITL flight.** All results are in simulation. The ROS 2 / PX4 bridge is written and syntax-checked; its numerical core is validated in the simulator loop, but the node itself has not been run against PX4.
- **Model fidelity.** Motor and aerodynamic parameters come from the PX4 Gazebo X500 model and the Crazyflie literature, not from identification of a physical vehicle; body-drag coefficients are engineering estimates. Rotor aerodynamics use the standard quasi-static $T = k_f \omega^2$ model (no inflow/advance-ratio dependence, no blade-element or vortex-ring-state effects); the ground effect is applied per rotor but ceiling/wall effects are not modelled. Battery dynamics are represented only by a per-episode $\omega_{\max}$ scale.
- **PhysX discretization.** PhysX integrates at 200 Hz with a first-order scheme; relative to an RK4 reference this causes 3 mm to 9 mm position error over 1.5 s of open-loop flight (Table 2), negligible for closed-loop control but relevant for open-loop prediction studies (use the reference integrator).
- **Vision.** Depth policies are trained on analytic ray-cast depth of primitive obstacles with a parametric stereo-noise model; rendered depth agrees to 7 mm median, but real sensor artefacts (edge holes, speckle, IR interference, motion blur) are not modelled. RGB/RGB-D pipelines are implemented and execute end to end, but procedural scenes lack the texture/lighting diversity needed for transferable RGB policies.
- **Evaluation statistics.** Each RL configuration was trained with a single seed (compute budget of an 8 GB consumer GPU); confidence intervals quantify evaluation noise, not seed-to-seed variability. Classical baselines use reasonable default gains, not systematically tuned gains, so the reported margins are upper bounds of the RL advantage.
- **Platform issues.** Isaac Lab v2.3.2 fails to initialize rigid bodies when a viewport is rendered (GUI, video); quadsim applies a version-gated runtime patch (Section 6.3.1). Interactive GUI and 4096-environment training do not fit simultaneously in 8 GB VRAM. The first launch with rendering enabled crashed once in the shader compiler (empty cache); later launches succeeded.

8 Roadmap toward real-world deployment

Prioritized by expected impact on a successful first real flight:

1. **PX4 SITL in the loop** (x500 gz model, identical parameters): run the exported hover and tracking policies through `quadsim_px4_bridge`; compare closed-loop responses with quadsim; calibrate the thrust mapping and the rate-loop bandwidth. Acceptance: hover RMS error within 2× of quadsim.
2. **System identification of the target vehicle**: mass and inertia (bifilar pendulum), k_f, k_m and motor time constants (thrust stand), rotor-drag coefficient (flight logs); update the preset and center the DR ranges on the identified values; retrain with several seeds.
3. **Latency identification** of the full pipeline (estimator, companion computer, uXRCE-DDS, PX4) and matching the delay distribution used in training; add estimator-in-the-loop training using recorded EKF2 error statistics.
4. **Hardware hover/tracking experiments** (tethered, then free flight in a motion-capture arena) with the safety layer (geofence, odometry watchdog, offboard-loss failsafe), logging with the same HDF5 schema for sim/real comparison.
5. **Depth-based navigation transfer**: record real depth frames, extend the depth noise model or add image-space randomization, distill the state/range teacher into the depth student (DAgger) to reduce sample cost, then test in a real obstacle course.
6. **Platform**: multi-seed training and hyper-parameter sweeps, teacher–student distillation, recurrent policies for implicit system identification, migration to Isaac Lab 3.x once stable.

9 Reproducibility

- Software: Isaac Sim 5.1.0 (pip), Isaac Lab v2.3.2 (`third_party/IsaacLab`), Python 3.11, torch 2.7.0+cu128, warp-lang 1.11.1, rsl-rl-lib 3.1.2, skrl 1.4.3, onnxruntime 1.20.1 (all pins in `env/constraints.txt`); native installation with `scripts/setup_env.sh`; container `docker/Dockerfile` on `nvcr.io/nvidia/isaac-lab:2.3.2`.
- Every training run stores the exact environment and agent configuration (YAML), command line, git commit and diff summary next to its checkpoints; evaluation JSONs store seeds and overrides.
- Validation: `pytest tests/core` (unit tests), `scripts/validation/m{1,2,3,6}_*.py` (milestone acceptance tests), `scripts/rl/evaluate.py` (policy evaluation); tables in this report are generated from `docs/results/*.json` by `scripts/tools/report_tables.py`.

References

[1] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, D. Scaramuzza. Champion-level drone racing using deep reinforcement learning. *Nature* 620, 2023.

[2] J. Eschmann, D. Albani, G. Loianno. Learning to fly in seconds. *IEEE RA-L*, 2024.

[3] E. Kaufmann, L. Bauersfeld, D. Scaramuzza. A benchmark comparison of learned control policies for agile quadrotor flight. *ICRA*, 2022.

[4] M. Kulkarni, W. Rehberg, K. Alexis. Aerial Gym Simulator: A framework for highly parallelized simulation of aerial robots. *IEEE RA-L*, 2025.

[5] B. Xu et al. OmniDrones: An efficient and flexible platform for reinforcement learning in drone control. *IEEE RA-L*, 2024.

[6] T. Lee, M. Leok, N. H. McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). *IEEE CDC*, 2010.

[7] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, P. Abbeel. Asymmetric actor critic for image-based robot learning. *RSS*, 2018.

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.

[9] J. Förster. System identification of the Crazyflie 2.0 nano quadrocopter. BSc thesis, ETH Zurich, 2015.